

Assignment (NumPy Fundamentals)

Daily Bicycle Rentals

Generative AI (GenAI) Use

GenAI Level 1 — Limited AI Use ⚠

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI (GenAI) Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Due date: Respect the due date stated on Moodle for this assignment.

Submission requirements (mandatory)

1. **Submit on Moodle as one compressed folder (.zip).**
 2. **An in-person presentation to the professor is required to receive a grade.** If you do not present, the assignment will **not** be graded, even if you submit it on Moodle.
 3. Your **report must include a comprehensive set of screenshots** demonstrating your practical work (see “Evidence and Screenshots Checklist”).
-

Scenario

You are a junior data analyst working for a city mobility team. They track **daily bicycle rentals** and need a **NumPy-based workflow** to:

- load numeric data from CSV files,
- reshape and combine datasets across years,
- extract subsets using slicing and boolean filtering,
- compute reliable summary statistics,
- export results for downstream reporting.

You must complete the tasks using **NumPy only**, following the concepts covered in the provided materials.

Dataset (provided)

Use the provided files (download and keep the raw copies unchanged):

- `tfl-daily-cycle-hires-2018.csv`
- `tfl-daily-cycle-hires-2019.csv`
- `warehouse_temperatures_jan.csv`

Important constraints (from course scope):

- Use `np.loadtxt(...)` for loading numeric data.
- These CSVs include headers; you must use the correct parameters (e.g., `delimiter`, `skiprows`, `usecols`).
- `np.savetxt(...)` can export **numeric arrays only**.

Deliverables (inside one .zip)

- **report.pdf** (written report with screenshots embedded)
- **notebook.ipynb** (recommended) or **script.py**
- **dataset/** folder containing:
 - unchanged raw CSV files (as provided)
 - your exported outputs (generated by NumPy)

Technical constraints (strict)

You may use only topics/tools present in the provided materials, including:

- Array creation: `np.array`, `np.full`, `np.arange`, `np.linspace`, `np.random.rand`, `np.random.randint`
- Shape inspection: `.shape`
- Data types: `dtype` (including long double examples if used)
- Indexing/slicing: 1D/2D slicing, reversing, `m[row_slice, col_slice]`

- Boolean indexing and its flattening behavior
- Reshape: `.reshape / np.reshape`
- Flatten/ravel: `.flatten, .ravel`
- Concatenation: `np.concatenate(...)` (axis rules)
- Transpose: `.T`
- Sorting: `np.sort(...)` (axis rules)
- Arithmetic and broadcasting, including scalar operations
- Aggregations: `np.sum, np.cumsum, np.mean, np.max, np.min, np.count_nonzero`
- I/O: `np.loadtxt, np.savetxt`

Do **not** use Pandas for this assignment.

Tasks

Part A — Load and validate numeric import (I/O)

1. Load the **hires** column from:
 - `tfl-daily-cycle-hires-2018.csv`
 - `tfl-daily-cycle-hires-2019.csv`
2. In your report, show:
 - the array `.shape` after loading,
 - evidence that the import is numeric (e.g., `dtype` or printed sample),
 - the reshape to **(365, 1)** for each year.

Requirement: Use `skiprows` and `usecols` correctly (column indexing starts at 0).

Part B — Combine years into a single matrix (concatenation + shape reasoning)

Create a combined dataset `tfl_1819` with shape **(365, 2)** where:

- column 0 = 2018 hires

- column 1 = 2019 hires

Requirement: Use `np.concatenate(..., axis=1)` and document **why axis=1** is correct.

Part C — Subsetting with slicing and orientation checks (indexing/slicing + transpose)

1. Extract a **30-day window** (e.g., days 100–129) from `tfl_1819` (both years).
 2. Create a **transposed** version of this 30-day window using `.T`.
 3. In the report, explain what transpose changes (rows ↔ columns) and show shapes before/after.
-

Part D — Boolean filtering and the “flattening” pitfall

Using `tfl_1819`, create a boolean mask for days where **2019 hires > 2019 mean**.

- Apply the mask to return the filtered values.

Requirement: In the report:

- show the boolean mask structure,
 - show the filtered result,
 - explicitly state and demonstrate that boolean indexing returns a **flattened 1D** array and changes `.shape`.
-

Part E — Vectorized arithmetic and broadcasting (no manual loops)

Compute daily year-over-year difference:

- `diff = (2019 hires) - (2018 hires)` using vectorized operations.

Then compute:

- `diff.mean()`
- `diff.max()` and `diff.min()`

Requirement: Explain (briefly) why these operations are vectorized and how broadcasting/shape compatibility enables them.

Part F — Summary analytics + export results (aggregation + savetxt)

1. Compute, for each year (column-wise):

- total hires (`sum(axis=0)`)
- average daily hires (`mean(axis=0)`)
- maximum daily hires (`max(axis=0)`)

2. Export:

- `tfl_1819` to a CSV output using `np.savetxt(...)`
- `diff` to a CSV output using `np.savetxt(...)`

Requirement: Show a screenshot proving the files were created and that the exported outputs are numeric.

Part G — Short creation/manipulation check (array creation + reshape + sort)

Using `warehouse_temperatures_jan.csv`:

1. Load the temperature column into a NumPy array.
2. Compute:
 - `count_nonzero, min, max, mean`
3. Create a sorted version using `np.sort(...)`.
4. Reshape the temperatures into a matrix of shape **(31, 1)** and show `.shape`.

Keep this part brief; it is intended to validate I/O + basic statistics on a simpler file.

Evidence and Screenshots Checklist (mandatory)

Your **report.pdf** must include screenshots showing:

1. Successful `np.loadtxt(...)` for each year + correct parameters used
2. `.shape` before and after reshaping to **(365, 1)**
3. Construction of `tfl_1819 + .shape == (365, 2)`

4. A slicing example (30-day window) + a transpose example with shapes
5. A boolean mask + filtered result + explicit proof of flattening behavior
6. Vectorized `diff` computation + summary statistics
7. Column-wise statistics (`sum/mean/max`) for the combined matrix
8. `np.savetxt(...)` outputs created (file listing or directory view)
9. Warehouse temperatures: load + basic stats + sorting + reshape evidence

If screenshots are missing or insufficient, the practical component is considered incomplete.

Presentation requirement (in-person)

You must present your work to receive a grade. Be prepared to:

- run your notebook/script,
 - explain how each NumPy requirement was met (shape reasoning, axis choices, masking behavior),
 - justify design choices (e.g., why `axis=1` for concatenation, why boolean filtering flattens).
-

Rubric (100 points)

1) Data loading and reshaping (20 pts)

- (10) Correct `np.loadtxt` usage (delimiter/skiprows/usecols) for both years
- (10) Correct reshape to (365,1) for each year with evidence

2) Combining datasets (concatenation + shapes) (15 pts)

- (10) Correct construction of (365,2) matrix using `np.concatenate(..., axis=1)`
- (5) Clear explanation of axis choice and shape compatibility

3) Indexing, slicing, transpose (15 pts)

- (8) Correct 30-day window extraction with proper 2D slicing

- (7) Correct transpose with shape explanation and evidence

4) Boolean indexing (flattening behavior) (15 pts)

- (8) Correct mask definition and application
- (7) Correct explanation and evidence of flattening + shape change

5) Vectorized arithmetic + summary stats (15 pts)

- (8) Correct vectorized `diff` computation (no manual loops)
- (7) Correct mean/min/max on `diff` with clear reporting

6) Aggregations and exports (15 pts)

- (8) Correct column-wise stats on `tfl_1819 (axis=0)`
- (7) Correct exports using `np.savetxt` and proof of output files

7) Temperatures mini-check (5 pts)

- (5) Correct load + stats + sort + reshape evidence

8) Report quality and screenshot completeness (10 pts)

- (6) Screenshots complete and aligned with checklist
- (4) Technical clarity, correct interpretation of outputs